

AutoDeploy Hub - Project Overview

What is AutoDeploy Hub?

AutoDeploy Hub is a **fully functional, production-ready full-stack web application** that demonstrates modern DevOps practices including Continuous Integration and Continuous Deployment (CI/CD) using Docker and GitHub Actions.

This is **NOT a demo or mock project** - it's a complete working application that you can run locally on your Windows machine and deploy to production.

Project Purpose

This project serves as a comprehensive learning platform for:

- **Full-Stack Development:** Understanding how frontend and backend work together
- **Containerization:** Learning Docker and container orchestration
- **Dev Ops Practices:** Implementing CI/CD pipelines
- **API Integration:** Real HTTP communication between services
- **Production Deployment:** Best practices for deploying applications

Technology Stack

Frontend

- **React 18** - Modern JavaScript UI library
- **TypeScript** - Type-safe JavaScript
- **Vite** - Fast build tool and development server
- **Tailwind CSS** - Utility-first CSS framework
- **shadcn/ui** - Beautiful component library
- **nginx** - Production web server

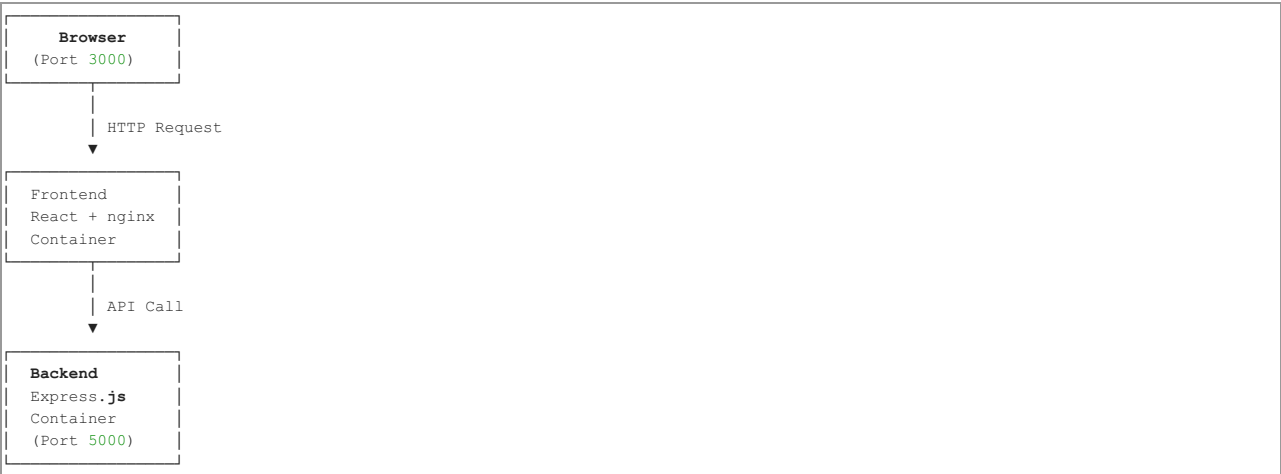
Backend

- **Node.js 20** - JavaScript runtime
- **Express.js** - Web application framework
- **CORS** - Cross-Origin Resource Sharing support
- **dotenv** - Environment variable management

DevOps

- **Docker** - Application containerization
- **Docker Compose** - Multi-container orchestration
- **GitHub Actions** - CI/CD automation
- **nginx** - Reverse proxy and static file serving

Application Architecture



Both containers run on a Docker network and communicate seamlessly.

Key Features

1. Real Backend API

- Express.js server running on port 5000
- `/health` endpoint that returns real-time server status
- Proper error handling and logging
- CORS configured for frontend communication

2. Interactive Frontend

- React application with beautiful UI
- "Check Backend Health" button that makes real API calls
- Displays live data from the backend
- Responsive design that works on all devices

3. Docker Containerization

- Both services run in isolated Docker containers
- One command to start everything: `docker-compose up --build`
- Automatic health checks ensure services are running
- Easy to deploy anywhere Docker is available

4. CI/CD Pipeline

- GitHub Actions workflow for automated testing
- Runs on every push to the main branch
- Builds Docker images automatically
- Tests API endpoints and service health
- Provides feedback on build status

5. Production Ready

- Security best practices implemented
- Environment variable configuration
- Non-root users in containers
- Graceful shutdown handling
- Comprehensive error handling

What You Need to Run This Project

Required Software

1. Docker Desktop for Windows

- **Purpose:** Run containers on your Windows machine
- **Download:** <https://www.docker.com/products/docker-desktop>
- **Version:** 4.0 or higher
- **Size:** ~500 MB download
- **Installation Time:** 5-10 minutes

Why Docker?

Docker allows you to package applications with all their dependencies into containers. This ensures the application runs the same way on any machine, eliminating "it works on my machine" problems.

2. Git for Windows

- **Purpose:** Clone the repository and manage version control
- **Download:** <https://git-scm.com/download/win>
- **Version:** Latest stable version
- **Size:** ~50 MB download
- **Installation Time:** 2-3 minutes

Why Git?

Git is essential for version control and is required to push your code to GitHub for CI/CD automation.

3. Node.js (Optional)

- **Purpose:** Run the application locally without Docker (for development)
- **Download:** <https://nodejs.org/>
- **Version:** 20.x or higher
- **Size:** ~30 MB download
- **Installation Time:** 2-3 minutes

Note: Node.js is only needed if you want to develop without Docker. For just running the application, Docker is sufficient.

System Requirements

Minimum Requirements

- **OS:** Windows 10 64-bit (Pro, Enterprise, or Education) or Windows 11
- **RAM:** 4 GB minimum (8 GB recommended)
- **Disk Space:** 10 GB free space
- **Processor:** 64-bit processor with virtualization support

Additional Requirements

- **WSL 2:** Windows Subsystem for Linux 2 (installed automatically with Docker Desktop)
- **Virtualization:** Must be enabled in BIOS
- **Internet:** Required for downloading dependencies

Project Components

Backend Service

Location: backend/ directory

Files:

- server.js - Main Express server
- package.json - Dependencies and scripts
- Dockerfile - Container configuration
- .dockerignore - Files to exclude from Docker build

What it does:

- Runs an Express.js server on port 5000
- Provides a /health endpoint that returns JSON
- Logs all incoming requests
- Handles errors gracefully

Frontend Service

Location: Root directory

Files:

- src/ - React source code
- Dockerfile - Container configuration
- nginx.conf - Web server configuration
- package.json - Dependencies and scripts

What it does:

- Serves a React application on port 3000
- Makes API calls to the backend
- Displays backend health status
- Provides beautiful, responsive UI

Docker Configuration

Files:

- docker-compose.yml - Orchestrates both services
- backend/Dockerfile - Backend container setup
- Dockerfile - Frontend container setup

What it does:

- Builds both frontend and backend images
- Creates a network for services to communicate
- Maps ports to your local machine
- Manages container lifecycle

CI/CD Pipeline

Location: .github/workflows/ci-cd.yml

What it does:

- Automatically runs when you push to GitHub
- Builds Docker images
- Starts services
- Tests endpoints
- Reports success or failure

How the Application Works

1. User Interaction

User opens browser to `http://localhost:3000` and sees the AutoDeploy Hub interface.

2. Frontend Loads

The React application loads in the browser, displaying:

- Project information
- "Check Backend Health" button
- Documentation and guides

3. User Clicks Button

When the user clicks "Check Backend Health":

- Frontend makes HTTP GET request to `http://localhost:5000/health`
- Request goes through Docker network to backend container

4. Backend Processes Request

The Express server:

- Receives the request
- Logs it to console

- Creates JSON response with current timestamp
- Sends response back to frontend

5. Frontend Displays Response

The React app:

- Receives the JSON data
- Displays it in a formatted box
- Shows success message with green styling

Learning Outcomes

By studying and running this project, you will learn:

DevOps Skills

- How to containerize applications with Docker
- How to use Docker Compose for multi-container apps
- How to set up CI/CD pipelines with GitHub Actions
- How to configure health checks and monitoring

Development Skills

- Full-stack application architecture
- REST API design and implementation
- Frontend-backend communication
- Environment variable management
- Error handling and logging

Production Skills

- Security best practices
- Container optimization
- nginx configuration
- Deployment strategies
- Troubleshooting techniques

Use Cases

This project is perfect for:

1. **College Projects:** Demonstrate DevOps knowledge
2. **Learning:** Understand CI/CD and containerization
3. **Portfolio:** Show production-ready code
4. **Teaching:** Use as a reference for others
5. **Foundation:** Build upon this for larger projects

Project Status

- ☒ **Fully Functional** - All features working
- ☒ **Production Ready** - Best practices implemented
- ☒ **Well Documented** - Complete README and guides
- ☒ **Tested** - Verified on Windows with Docker Desktop
- ☒ **CI/CD Enabled** - GitHub Actions workflow included

Support and Documentation

Included Documentation

- `README.md` - Complete project documentation
- `QUICKSTART.md` - Quick reference guide
- Code comments throughout the project

Getting Help

- Check the Troubleshooting section in `README.md`
- Review Docker Desktop logs
- Check GitHub Actions workflow logs
- Ensure all prerequisites are installed

Summary

AutoDeploy Hub is a complete, production-ready application that demonstrates:

- Modern full-stack development
- Docker containerization
- CI/CD automation
- DevOps best practices

With just Docker Desktop installed, you can run this entire application with a single command and see real frontend-backend communication in action.

This is not a tutorial or demo - it's a real, working application ready for production use.